1. Neural-network basics

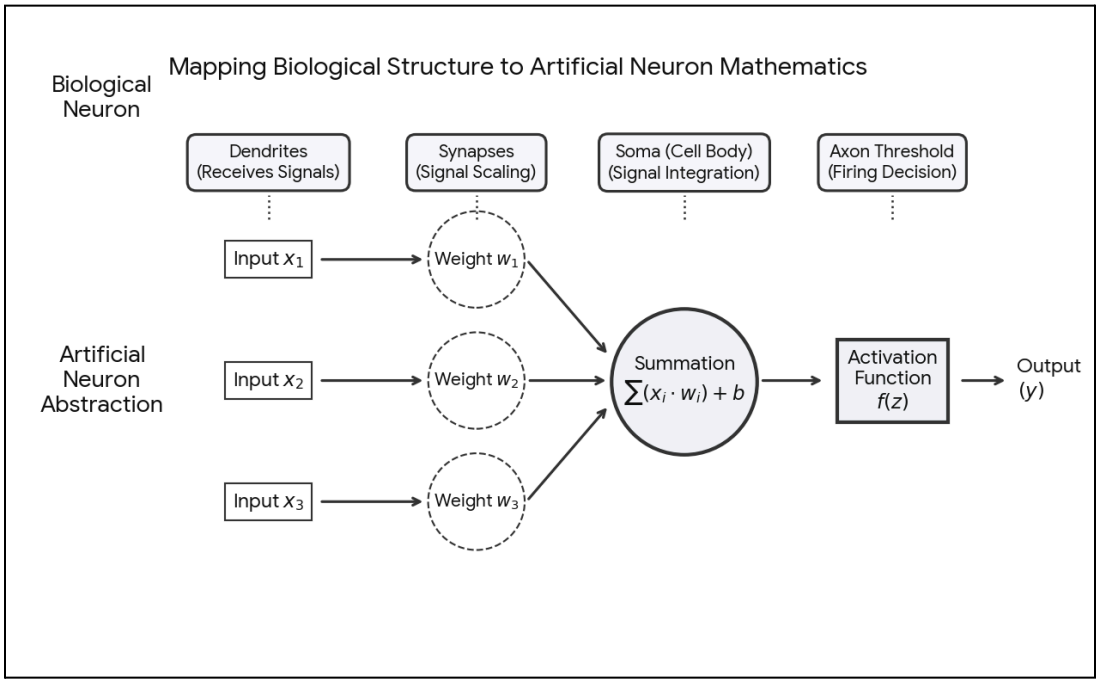


Fig.a. Relation between Biological Neuron and Artificial Neuron

Output of a neuron (y) is transmitted to other neuron through Axon ; Inputs of a neuron are received from other neurons by Dendrite.

Neuron :

A neuron is a **computational element** that computes the weighted sum (dot product) of its input activations and corresponding weights, adds a bias, applies an activation function, and produces a scalar output.

$$y = f\left(\sum_i x_i w_i + b\right)$$

Fig. Function of a neuron

x - input activation vector

W - weight vector

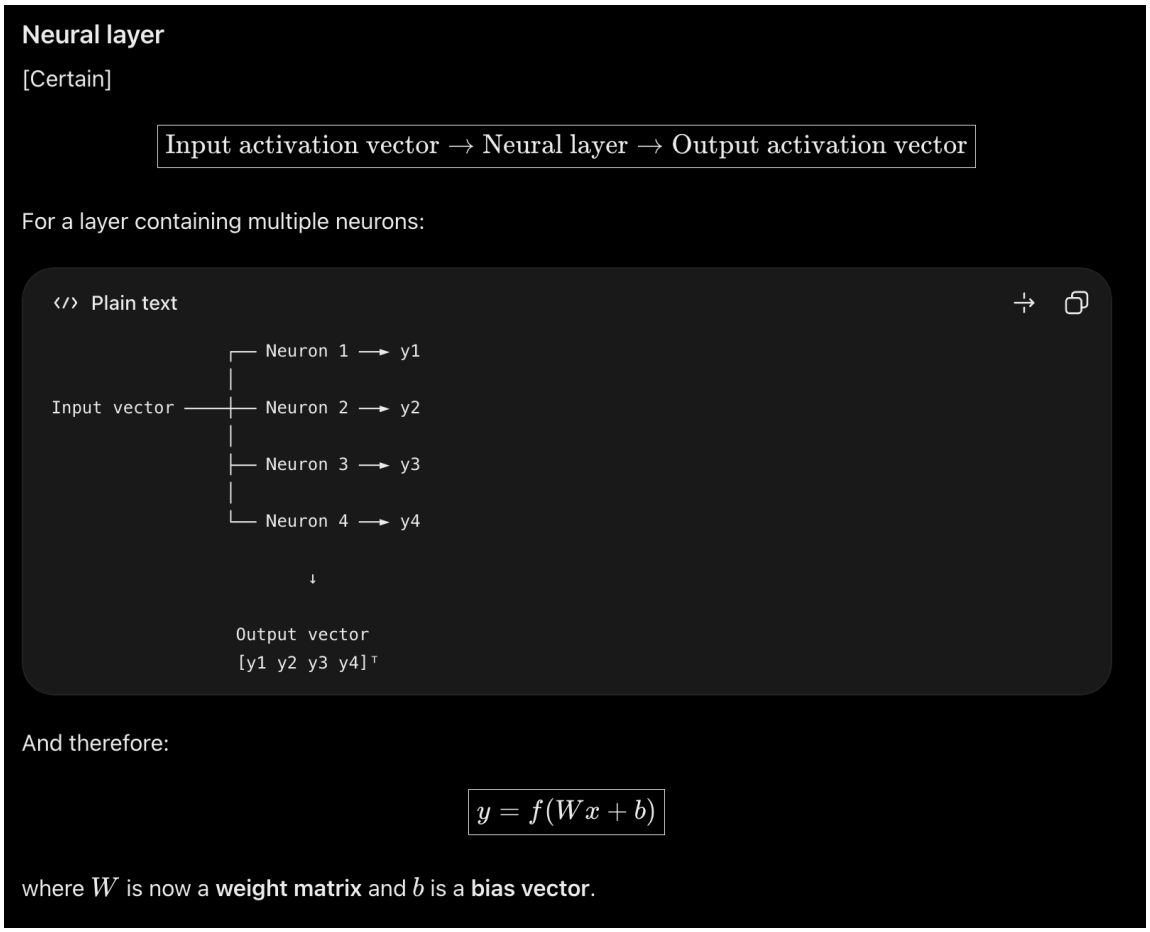
b - scalar bias

y - output activation scalar

A neural layer is a collection of neurons that transforms an input activation vector into an output activation vector.

N.B.

1. A neuron basically transforms an input activation vector into scalar output ; A neural layer transforms an input activation vector into output activation vector.



x - input activation vector

W - weight matrix

b - bias vector

y - output activation vector

Activation :
A numerical value representing the output of a neuron after its activation function; the output activations of one layer become the input activations of the next layer.

Example:

x = [2, -1, 4, 3]

These are input activations.

y = [1,0,-8,4]

Weight :

A learned coefficient multiplied by an activation. It actually indicates how much does an input contribute to the summation.

Example :

x0 × w0

x1 × w1

x2 × w2

...

Bias :

A constant added after the accumulated products. It moves the **dot product value** up or down. Its value determine the **point at which a neuron** can **respond**.

acc = Σ(x[i] × w[i])

result = acc + bias

Example :

Let's say we have an input activation
x = temperature

and we want:

Temperature <= 50°C → normal

Temperature > 50°C → abnormal

Suppose w = 1 , b = -50

Then Summation,z = x - 50 // This summation can also be called PRE-ACTIVATION

Now output , y = f(summation)

Assume y = RELU(x-50) ; [RELU - REctified **L**inear **U**nit**] ; Meaning of RELU : If z <= 0 , y = 0 ; If z > 0 , y = summation**

4. Examples

Temperature x	$z = x - 50$	ReLU y	Interpretation
30°C	-20	0	Normal
40°C	-10	0	Normal
50°C	0	0	Normal
55°C	5	5	Abnormal
60°C	10	10	Abnormal

MAC (**M**ultiply **A**nd **A**ccumulate)

Multiply-Accumulate is the fundamental operation:

$$ACC_{NEW} \leftarrow ACC_{OLD} + X \times W$$

Now see why this matters for an NPU

[Certain] A neural-network neuron essentially needs this operation:

$$y = x_0w_0 + x_1w_1 + x_2w_2 + x_3w_3 + b$$

The NPU can perform:

Plain text

```
ACC = 0

ACC = ACC + X0 × W0
ACC = ACC + X1 × W1
ACC = ACC + X2 × W2
ACC = ACC + X3 × W3

ACC = ACC + BIAS
```

2. Neural-network workload terminology

Dot Product :

A dot product is:

$$x \cdot w = \sum_i x_i w_i$$

Example :

x = [1,2,3]

w = [4,5,6]

1×4 + 2×5 + 3×6 = 32

Our first NPU will essentially be a hardware dot-product engine controlled by microcode.

Vector :

An ordered collection of numbers ; Each number is called scalar

Based on the orientation of elements , it is classified as

Column vector : N rows and 1 column

Example :

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix} \quad \rightarrow \quad \text{column vector}$$

Row vector : 1 row x N columns

$$x = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix} \quad \rightarrow \quad \text{row vector}$$

For vectors:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$$

has shape:

$$\boxed{3 \times 1}$$

while:

$$x^T = \begin{bmatrix} 1 & 2 & 3 \end{bmatrix}$$

has shape:

$$\boxed{1 \times 3}$$

Matrix :

A 2-D collection of values.

Example :

[1 2 3]

[4 5 6]

Shape of matrix or vector :

Number of rows x Number of columns

Matrix-Vector Multiplication : (Collection of dot products ; The result is a column vector or row vector)

This becomes y = Wx , where W is called Weight Matrix , x is called activation vector

For example:

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \qquad x = \begin{bmatrix} 10 \\ 20 \\ 30 \end{bmatrix}$$

Then:

$$y = Wx$$

gives:

$$y = \begin{bmatrix} 1(10) + 2(20) + 3(30) \\ 4(10) + 5(20) + 6(30) \end{bmatrix} = \begin{bmatrix} 140 \\ 320 \end{bmatrix}$$

Each row of the weight matrix is associated with one output neuron.

In the above example , the output vector y is column vector

N.B. For a neuron

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

w1 , w2 , w3 can have the following characteristics

1. Can have different values : w1 = 2 , w2 = 5 , w3 = -1

2. Can have same values : w1 = w2 = w3 = 2

3. Can have positive or negative values

4. Zero , meaning that input does not contribute to the summation

3. Layer terminology

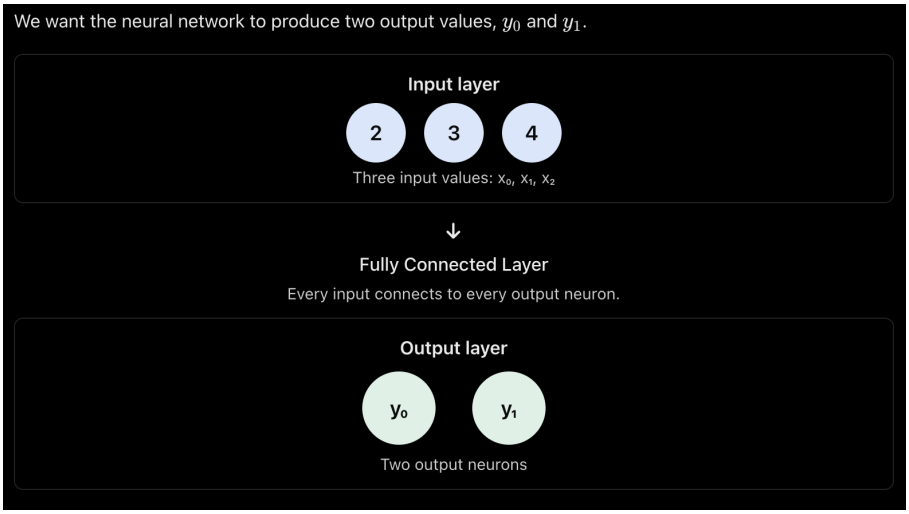
Fully Connected or Dense Connected Layer :

Every input connects to every output neuron

Example :

1. Imagine a neural network with 3 inputs and 2 output neurons

Assume the input activation vector ,X = [2,3,4] ; x0 = 2 , x1 = 3 , x2 = 4



Because there are three inputs and two outputs, the layer requires six weights: three for each output neuron.

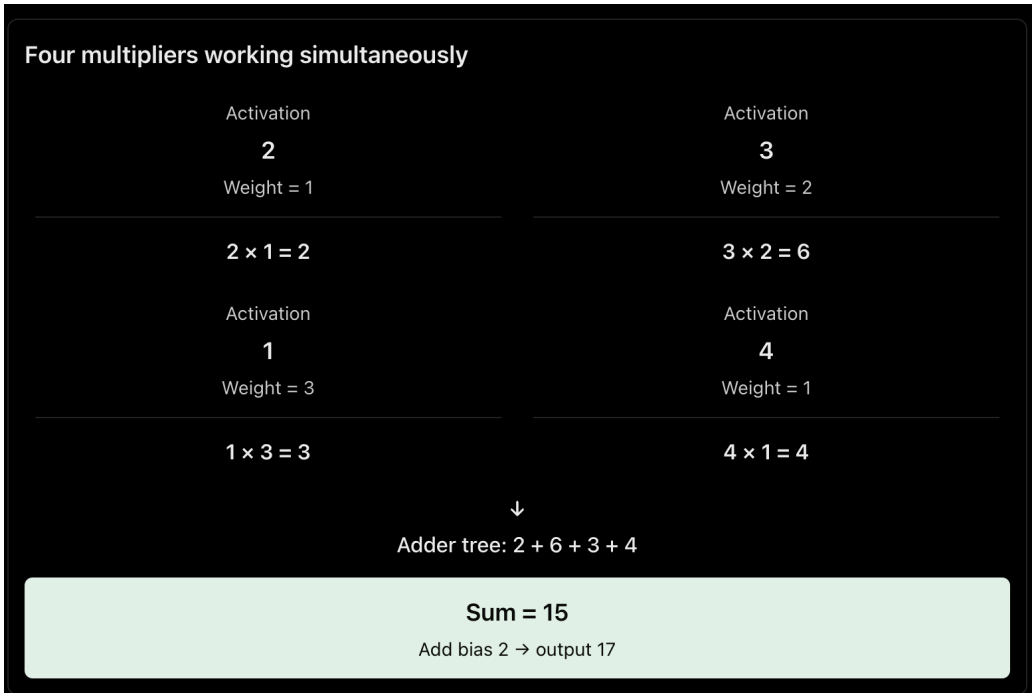
The weights determine how strongly each input contributes to each output.

N.B. The **input activation vector** to a **neuron** need not **necessarily come from a previous neural layer** ; They can also originate from raw sensor data, image pixels, or values loaded from memory for testing. The **neuron** hardware can **receive the** values of **input activation vector** simultaneously or **sequentially** (**one by one**)

Case 1 : Parallel Computation

$$\mathbf{x = [x_0 , x_1 , x_2 , x3] \ , w = [w0 , w1 , w2 , w3]}$$

In a parallel implementation, we provide all four activations and all four corresponding weights to the computation hardware at the same time.



Four multipliers calculate the four products simultaneously. An adder tree then combines the products, after which the bias is added.

What hardware do we need?

Four multipliers, addition hardware, and sufficient registers or memory bandwidth to supply the operands together.

This can reduce computation latency, although the actual number of clock cycles depends on the implementation, memory timing, and whether the design is pipelined.

Case 2 : Sequential Computation

$$x = [x0 , x1 , x2 , x3] \quad w = [w0 , w1 , w2 , w3]$$



In this , one pair of activation and weight are taken at a time and they are applied to a MAC ; The bias 2 is a sample value for illustration purpose

So a single MAC is reused four times

N.B. A parallel implementation generally requires more multiplier hardware and operand bandwidth, while a sequential implementation reuses the same arithmetic unit.

2. Understand what one output neuron does

Let us first calculate y0

Assume the weights are

w00 = 1 ; w01 = 2 ; w02 = 3

Assume bias , b0 = 1

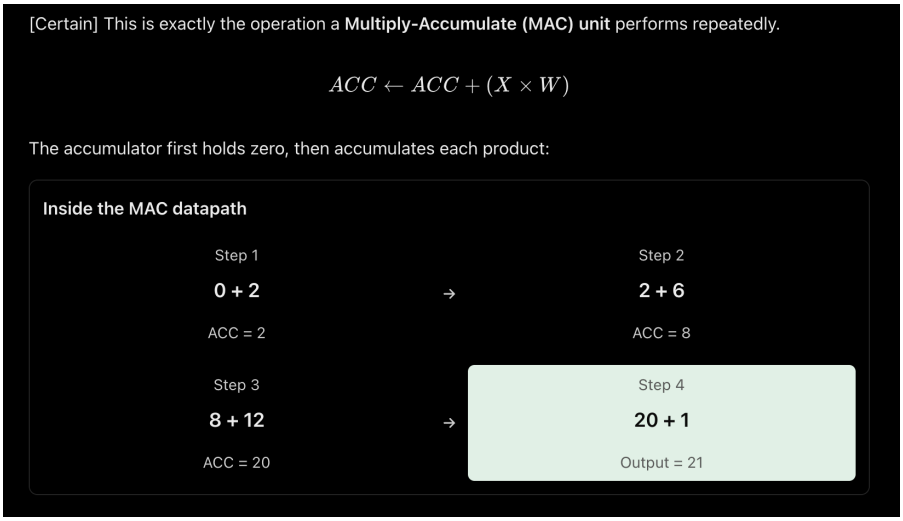
The formula is:

$$y_0 = \sum_i x_i w_{0i} + b_0$$

The symbol $\sum$ means *add all the following terms together*.

Substitute the values:

y0 = (2 x 1) + (3 x 2) + (4 x 3) + 1 = 2 + 6 + 12 + 1 = 20 + 1 = 21



Initially ACC on RHS = 0 , then we keep on updating ACC on LHS by the above expression

Now calculate y1

Assume the weights as

w10 = 2 , w11 = 1 , w12 = -1

Assume bias b1 = 2

Subsitute the values :

y1 = (2x2) + (3x1) + (4x-1) + 2 = 4 + 3 - 4 + 2 = 5

Notice something important: the inputs are the same, but the weights and bias are different. That is why the output neurons produce different results.

4. What does the equation $y_j = \sum_i x_i w_{ji} + b_j$ actually mean?

Let's decode every symbol.

Symbol	Meaning
x_i	The i-th input
w_{ji}	Weight connecting input i to output neuron j
b_j	Bias belonging to output neuron j
y_j	The calculated value of output neuron j
$\sum_i$	Multiply each input by its corresponding weight and add all products

[Certain] The first subscript in w_{ji} identifies the output neuron; the second identifies the input.

For our example, the weight matrix is:

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 1 & -1 \end{bmatrix}$$

The first row calculates y_0, while the second row calculates y_1.

This can also be written as a matrix operation:

$$Y = WX + b$$

where X is a column vector and b contains one bias per output neuron.

Convolution :

A convolution applies a small kernel(or filter matrix) repeatedly across an input feature map.

Input feature map
↓
convolution
↓

Output feature map

Concept of feature map :

A **feature map** is a grid of numbers representing a particular feature detected or represented in the input data. The **small input matrix window in the input matrix** over which the **filter** operates **is called input feature map** . The **final output matrix after the filter traversed all the positions** of the **input matrix** is called **output feature map**.

1. Start with an image

Suppose a computer receives this small grayscale image. Each number represents the brightness of one pixel.

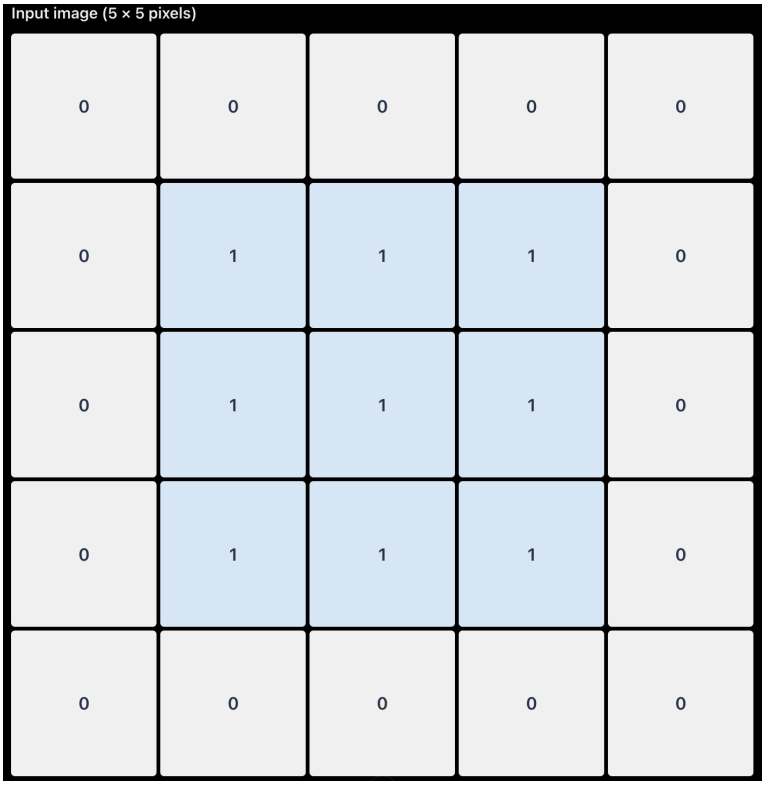
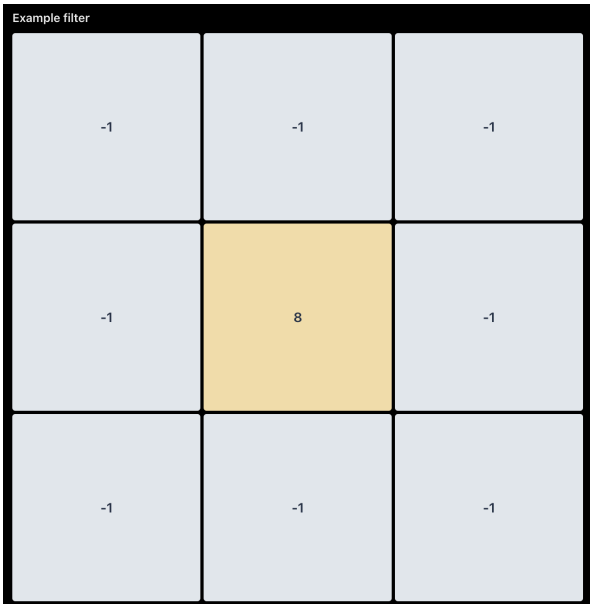


Fig.12 Here, **1** represents a **bright pixel** and **0** represents a **dark pixel**. The **bright pixels form a small square-like pattern.**

At this stage, this is the input image represented as numerical data. It is not yet a feature map generated by a convolution.

2. Apply a filter to detect a feature

Suppose the CNN uses a 3×3 **filter** (also called a **kernel**) to detect a particular pattern. Each **value** in this **filter matrix** is called **weight**.



Notice that the centre weight is **+8**, while the other eight weights are **-1** . This **filter responds strongly when the centre pixel is brighter than its surrounding pixels**. It is an illustrative centre-surround filter, not a trained CNN filter.

3. **Start applying the filter from top left portion and keep shifting one position right**

Position 1 — take the top-left window

The window contains:

$$X = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

The filter contains:

$$W = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

Multiply corresponding positions

Every pixel is multiplied by the filter weight at the same position.

$$\begin{aligned} S = &(0 \times -1) + (0 \times -1) + (0 \times -1) \\ &+ (0 \times -1) + (1 \times 8) + (1 \times -1) \\ &+ (0 \times -1) + (1 \times -1) + (1 \times -1) \end{aligned}$$

Now simplify:

$$S = 0 + 0 + 0 + 0 + 8 - 1 + 0 - 1 - 1$$

$$\boxed{S = 5}$$

[Certain] **The first output value is 5.** We have processed one window and generated one number.

Fig.a. **Top left input feature map**

Position 2 — slide the filter one pixel to the right

[Certain] We now move the entire filter one column to the right. The filter weights do not change; only the image pixels underneath them change.

The new window is:

$$X = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

The multiplication is:

$$S = 0 + 0 + 0 - 1 + 8 - 1 - 1 - 1 - 1$$

$$\boxed{S = 3}$$

Why did the result change from 5 to 3?

The new window contains more surrounding bright pixels. Each of those pixels contributes -1, reducing the total response.

Position 3 — slide right again

The next window is:

$$X = \begin{bmatrix} 0 & 0 & 0 \\ 1 & 1 & 0 \\ 1 & 1 & 0 \end{bmatrix}$$

Its centre pixel is still 1, but only three surrounding pixels are bright.

$$S = 8 - (1 + 1 + 1)$$

$$\boxed{S = 5}$$

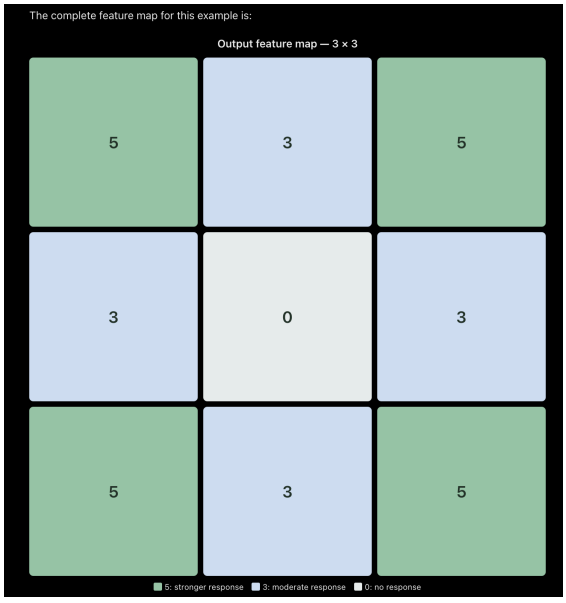
We have now generated three output values from the first three filter positions:

$$[5,\ 3,\ 5]$$

Repeat for every position (i.e. again reset your position to left most and then incrementally move towards right most)

Our input image is 5×5, and our filter is 3×3. If we move the filter one pixel at a time without padding the image, it fits at three horizontal positions and three vertical positions.

That gives us $3 \times 3 = 9$ output values.



The positive contribution of the centre pixel is exactly cancelled by the negative

contributions of the surrounding pixels.

Why did we design the filter this way?

The filter is designed to compare the centre pixel against its surroundings.

- **If the centre is bright and its neighbours are dark, the output is strongly positive.**
- **If the centre and all its neighbours are equally bright, the output is zero.**
- **If the centre is dark and its neighbours are bright, the output is negative.**

For example, if the centre pixel is 1 and all eight neighbours are 0, the output is $+8$.

Pixel values × filter weights

1 × -1 -1	1 × -1 -1	1 × -1 -1
1 × -1 -1	1 × 8 +8	1 × -1 -1
1 × -1 -1	1 × -1 -1	1 × -1 -1

Yellow: centre pixel. Blue: eight surrounding pixels.

Filter applied to the centre window in 5 x 5 image . The blue colored portion in Fig.12 is the central window

2. Calculate each contribution

[Certain] For the centre pixel:

$$1 \times 8 = +8$$

For each surrounding pixel:

$$1 \times (-1) = -1$$

There are eight surrounding pixels, so their combined contribution is:

$$(-1) + (-1) + (-1) + (-1) + (-1) + (-1) + (-1) + (-1) = -8$$

3. Add the contributions

Now combine the centre and surrounding pixels:

$$S = +8 + (-8)$$

$$\boxed{S = 0}$$

Think of it as having ₹8 and then spending ₹8. Your net balance becomes zero.

The positive contribution of the centre pixel is exactly cancelled by the negative contributions of the surrounding pixels.

ReLU (Rectified Linear Unit):

$$ReLU(x) = max(0,x)$$

Hardware is essentially:

if x < 0

y = 0;

else

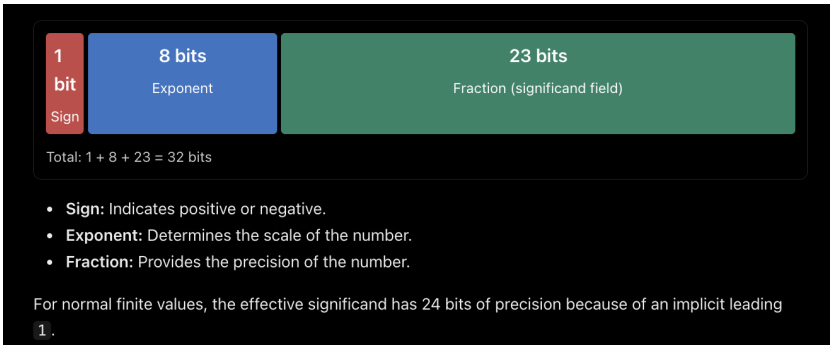
y = x;

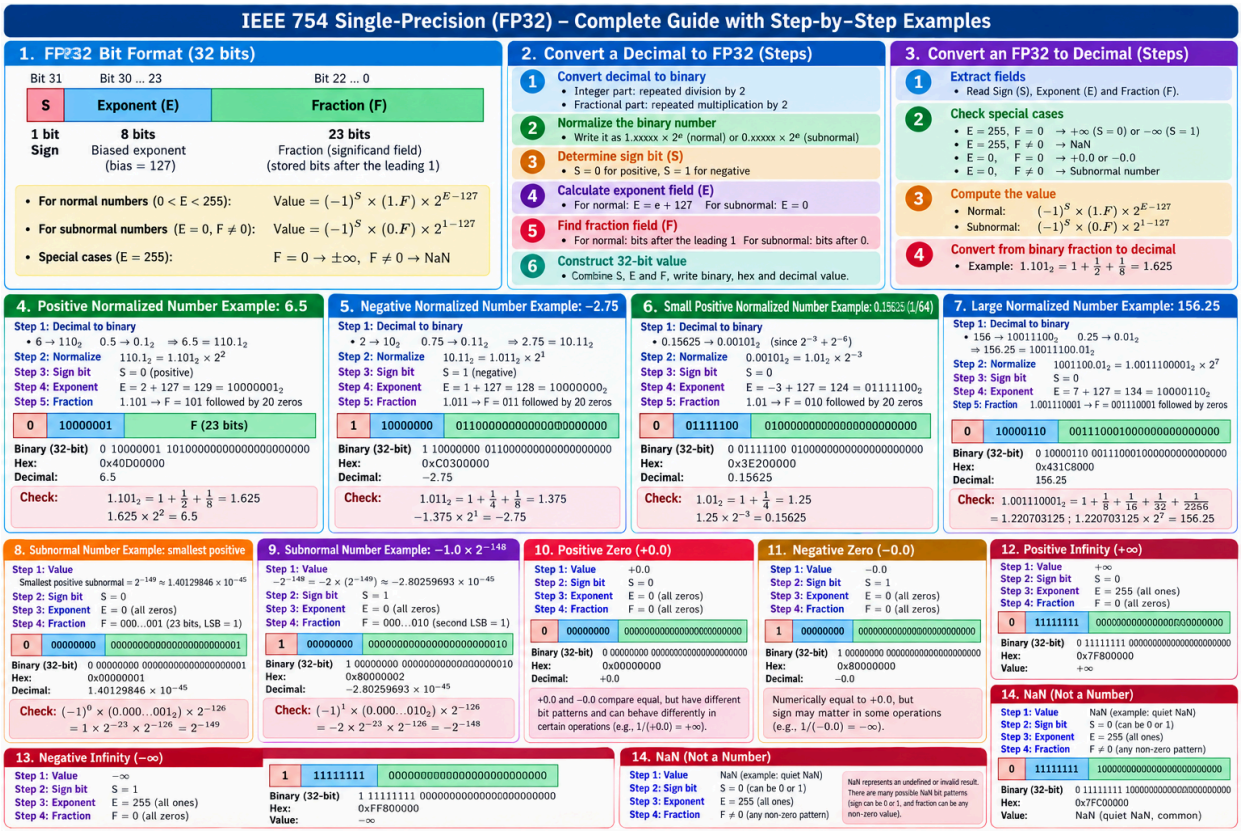
4. Numerical representation

INT8 - Refers generically to 8-bit integer ; The interpretation depends on the design

Representation	Range	Example bit pattern `1111_1111`
Signed INT8 (two's complement)	−128 to +127	−1
Unsigned UINT8	0 to 255	255

FP32 - IEEE 754 Single Precision 32-bit Floating Point Representation - FP32 can represent fractional values such as `3.14159`, `-0.75`, and `0.001`.





The exponent value , e , in 2^e format , refers to number of positions you are moving left (where e becomes positive) or number of positions you are moving right(where e becomes negative) the decimal point , in order to obtain NORMALIZED format (i.e. 1.F format)

e = E - 127 ; From the value of e , we determine the value of E(i.e. Exponent field in FP32 representation) i.e E = e + 127

For example consider 10011100.01_2

To normalize this (i.e. to convert to 1.F format) , move the decimal point left by 7 positions ; Therefore it becomes 1.001110001 x 2^7.

So in this case , 7 = E - 127 → E = 7 + 127 = 134.

Quantization :

Converting higher-precision neural-network values into lower-precision representations such as INT8.

Quantization in Neural Networks – Concepts, Terms, Formulas and Numerical Examples (Updated)

1. What is Quantization?

- Converts high-precision real values (usually floating point, e.g., FP32) into low-precision discrete integer values (e.g., INT8) for efficient neural network inference and training on hardware.
- Maps a continuous range of real numbers to a finite set of integer values.
- Involves scaling, rounding and (if needed) clipping (saturation).



2. Key Terms in Quantization

Term	Short Definition
Scale (s)	Factor that maps between real values and integer values. Real number > 0.
Zero-point (z)	Integer value that represents real zero. Can be zero (symmetric) or non-zero (asymmetric).
Quantization range	Minimum and maximum integer values (e.g., -128 to 127 for INT8).
Quantization step (Δx)	Smallest change in real value represented by 1 integer step. Equal to the scale (s).
Rounding	Method used to convert a fractional value to an integer (e.g., round to nearest, round up, round down, etc.).
Clipping (saturation)	Values outside the representable range are limited to the minimum or maximum value.

3. Quantization and Dequantization Formulas

3.1 Quantization (float → integer)

$$q = \text{round}\left(\frac{x}{s}\right) + z$$

- q : quantized integer value (e.g., INT8)
- x : real (floating point) value
- s : scale (positive real number)
- z : zero-point (integer, can be 0 or non-zero)
- $round(\ )$: chosen rounding method (e.g., round to nearest, up, down, etc.)

3.2 Dequantization (integer → float)

$$\hat{x} = (q - z) \times s$$

- $\hat{x}$: approximate real value
- q : quantized integer value
- s : scale
- z : zero-point

4. INT8 Range and Quantization Step

- Signed INT8 can represent 256 integer values: $-128, -127, \ldots, -1, 0, 1, \ldots, 126, 127$
- Total values = 256
- Minimum = -128 Maximum = 127
- For a real range $[x_{min}, x_{max}]$, scale is:

$$s = \frac{\hat{x}_{max} - z_{min}}{q_{max} - q_{min}}$$

Quantization step (Δx) = s

For INT8:
$q_{min} = -128$, $q_{max} = 127$

Examples of scale (step size)

Real range $[x_{min}, x_{max}]$	Scale s (Δx)
$[-1, 1]$	0.007843
$[0, 1]$	0.003922
$[0, 6]$	0.023529
$[-6, 6]$	0.047059

Note: Any quantized value outside $[-128, 127]$ is clipped (saturated) to -128 or 127.

5. Rounding Methods with Numerical Examples

Method	Definition	Example (x = 2.3)	Example (x = −2.3)	Example (x = 2.5)	Example (x = −2.5)
Round to nearest (half up)	Round to nearest integer. If exactly 0.5, round up (ties to $+\infty$).	2	−2	3	−2
Round to nearest even (banker's)	Round to nearest integer. If exactly 0.5, round to even integer.	2	−2	2	−2
Round up (ceiling)	Always round towards $+\infty$.	3	−2	3	−2
Round down (floor)	Always round towards $-\infty$.	2	−3	2	−3
Round towards zero (truncate)	Discard fractional part.	2	−2	2	−2
Round away from zero	Always move away from zero.	3	−3	3	−3

6. Symmetric Quantization (z = 0) – Numerical Example

- Zero-point $z = 0$.
- Real range is symmetric: $[-1.0, 1.0]$.
- Scale: $s = (1.0 - (-1.0))/(127 - (-128)) = 2.0/255 = 0.007843$.
- Formula: $q = \text{round}(x/s)$ (since $z = 0$).

Example: Quantize x = 0.3 (symmetric)

1. Scale: $\frac{x}{s} = 0./0.007843 = 38.28$
2. Rounding (nearest, half up): $\text{round}(38.28) = 38$
3. Add zero-point: $q = 38 + 0 = 38$ (INT8)
4. Dequantization: $\hat{x} = 38 \times 0.007843 = 0.2980$
5. Quantization error: $0.3 - 0.2980 = 0.0020$

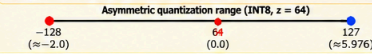


7. Asymmetric Quantization (z ≠ 0) – Numerical Example

- Zero-point z can be non-zero.
- Real range can be any $[x_{min}, x_{max}]$ (usually non-negative for activations).
- Example real range: $[0.0, 6.0]$.
- Scale: $s = (6.0 - 0.0)/(127 - (-128)) = 6.0/255 = 0.023529$.
- Zero-point: $z = round(0 - x_{min}/s) = round(0 - 0./0.023529) = 0$.
  To show non-zero z, consider a different range, e.g., $[-2.0, 6.0]$.

Example: Quantize x = 1.5 with range [−2.0, 6.0] (asymmetric)

1. Scale: $s = (6.0 - (-2.0))/255 = 8.0/255 = 0.031373$
2. Zero-point: $z = round(0 - (-2.0)/0.031373) = round(63.75) = 64$
3. Scale and round: $\frac{x}{s}\ s = 1.5/0.031373 = 47.79 \rightarrow round(47.79) = 48$
4. Add zero-point: $q = 48 + 64 = 112$ (INT8)
5. Dequantization: $\hat{x} = (112 - 64) \times 0.031373 = 1.5059$
6. Quantization error: $1.5 - 1.5059 = -0.0059$



8. Dequantization – Numerical Examples

Example 1: Symmetric case (z = 0)

- Given: $q = -64$, $s = 0.007843$, $z = 0$
- $\hat{x} = (q - z) \times s$
  $= (-64 - 0) \times 0.007843$
  $= -64 \times 0.007843 = -0.5020$

Dequantized value: $\hat{x} = -0.5020$

Example 2: Asymmetric case (z = 64)

- Given: $q = 112$, $s = 0.031373$, $z = 64$
- $\hat{x} = (q - z) \times s$
  $= (112 - 64) \times 0.031373$
  $= 48 \times 0.031373 = 1.5059$

Dequantized value: $\hat{x} = 1.5059$

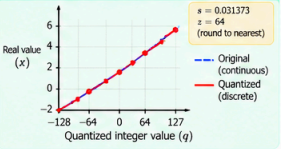
9. Quantization Examples for Multiple Values

(Asymmetric: [−2.0, 6.0], s = 0.031373, z = 64)

Real value x (FP32)	x / s	Rounded q (INT8)	Rounding method	q + z (quantized)	Dequantized $\hat{x} = (q - z) \times s$	Quantization error
−2.0	0.00	0	Exact	64	−2.0000	0.0000
−1.0	31.88	32	Nearest	96	−1.0039	0.0039
0.0	63.75	64	Nearest	128	0.0000	0.0000
1.0	95.63	96	Nearest	160	1.0039	−0.0039
1.5	143.25	48	Nearest	112	1.5059	−0.0059
3.0	159.25	95	Nearest	159	2.9765	0.0235
5.0	223.25	95	Nearest	223	4.9765	0.0235
6.0	255.00	127	Exact	191	5.9765	0.0235

10. Visual Illustration of Quantization

(Asymmetric: [−2.0, 6.0], s = 0.031373, z = 64)



11. Key Takeaways

1. Quantization maps continuous real values to discrete integer values using scale (s) and zero-point (z).
2. Quantization step (Δx) is equal to the scale (s).
3. Different rounding methods give different integer results, especially for negative values and exact 0.5 cases.
4. Values outside the representable range are clipped (saturated).
5. Symmetric quantization uses $z = 0$ and is common for weights.
6. Asymmetric quantization allows $z \neq 0$ and is common for activations.
7. Dequantization uses the same scale and zero-point to recover an approximate real value: $\hat{x} = (q - z) \times s$.

1. Symmetric quantization

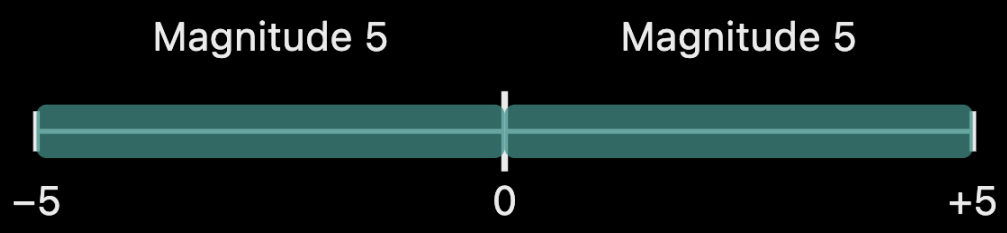
Suppose the chosen range is:

$$x_{\max} = +5, \qquad x_{\min} = -5$$

Therefore,

$$|x_{\max}| = |x_{\min}| = 5$$

Symmetric range

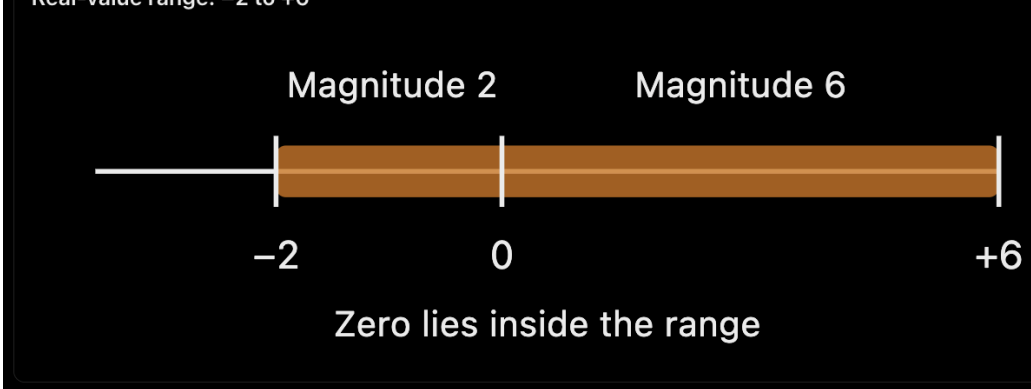


[Certain] The range is centered on zero, so the negative and positive limits have equal magnitudes.

Asymmetric Quantization :

$$\boxed{x_{\min} = -2, \qquad x_{\max} = +6}$$

Here, $|-2| = 2$ and $|+6| = 6$, so the range is not symmetric about zero.



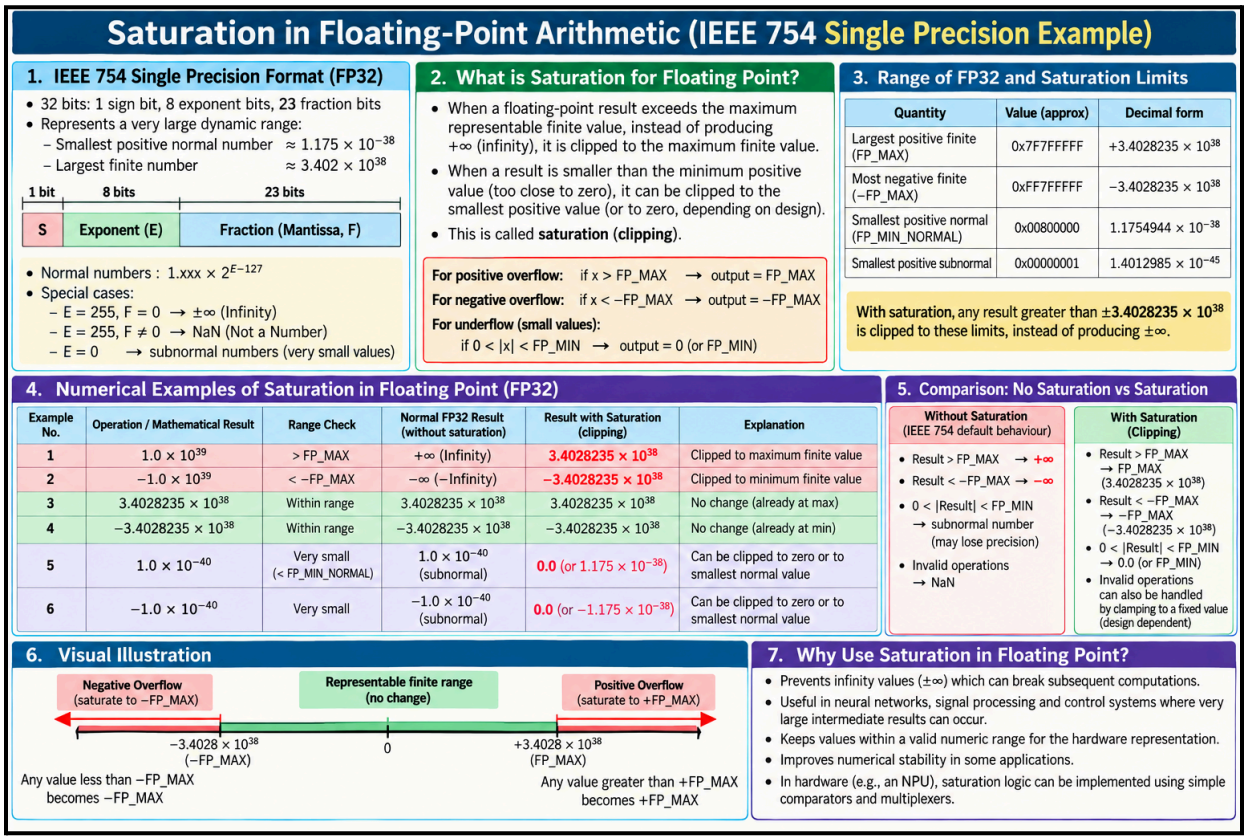
[Certain] For this example:

- Minimum real value: -2
- Maximum real value: $+6$
- Range width: $6 - (-2) = 8$

In symmetric quantization, negative and positive limits should have equal magnitudes. In asymmetric quantization, negative and positive limits do not need equal magnitudes.

Saturation or Clipping :

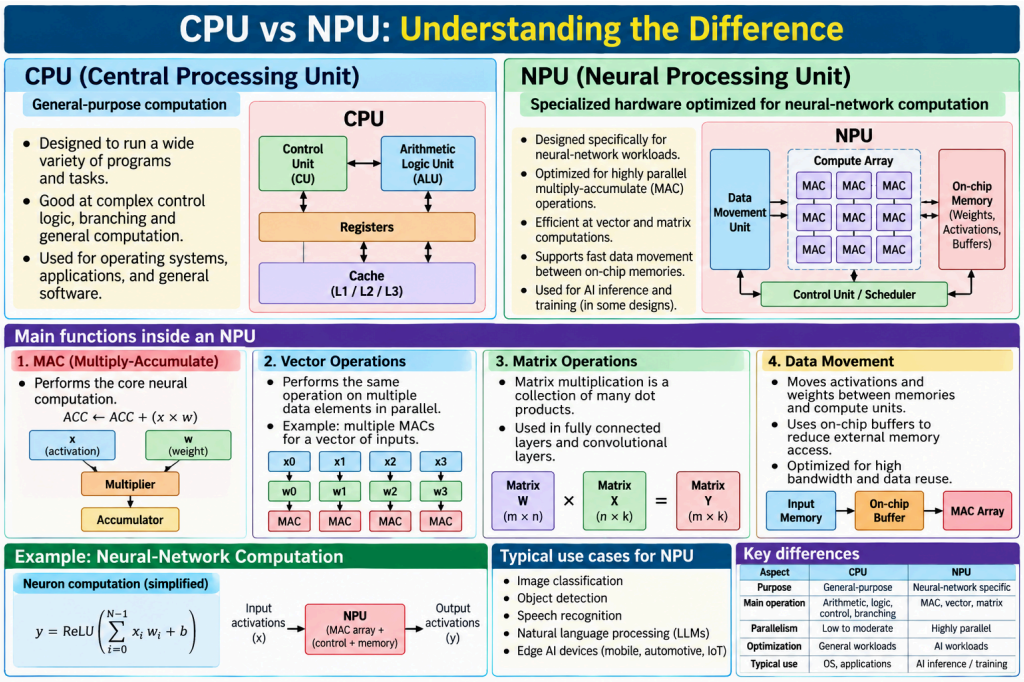
When a result exceeds the representable range, it is clipped to the allowed range.



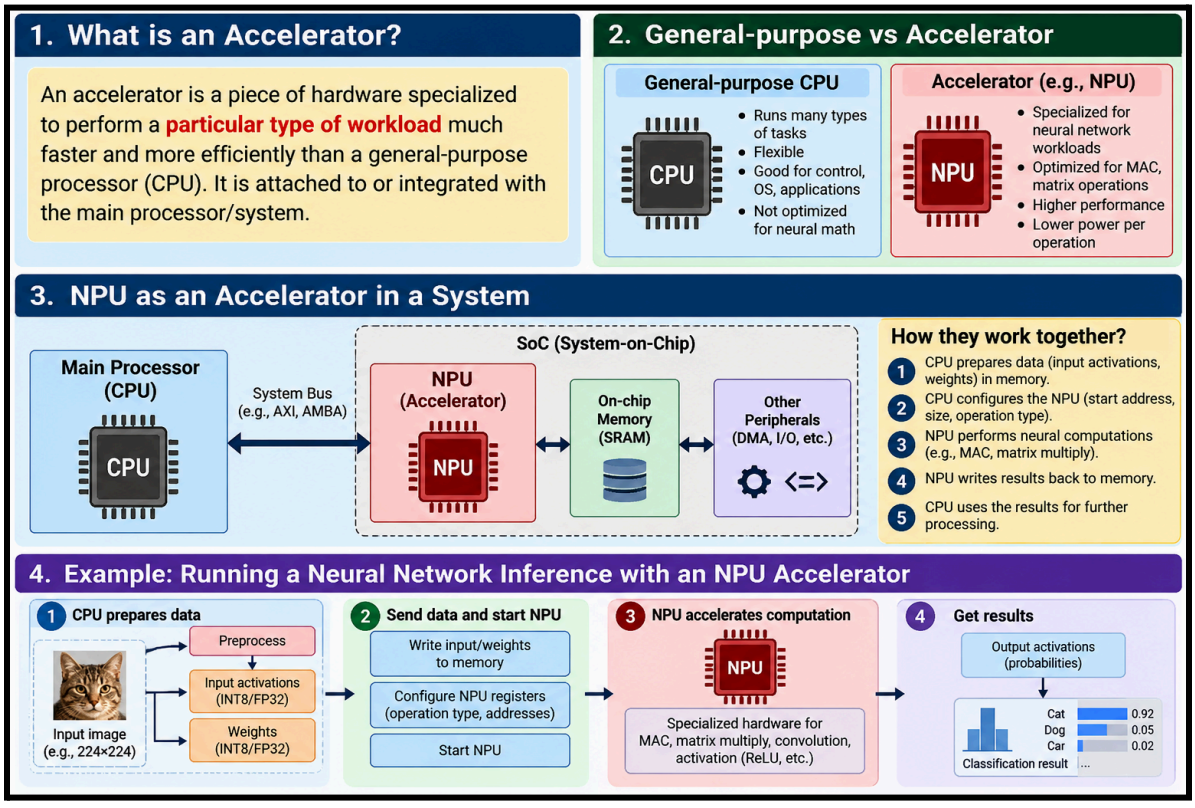
5. NPU architectural terminology

NPU

[Certain] Neural Processing Unit — specialized hardware optimized for neural-network computation.

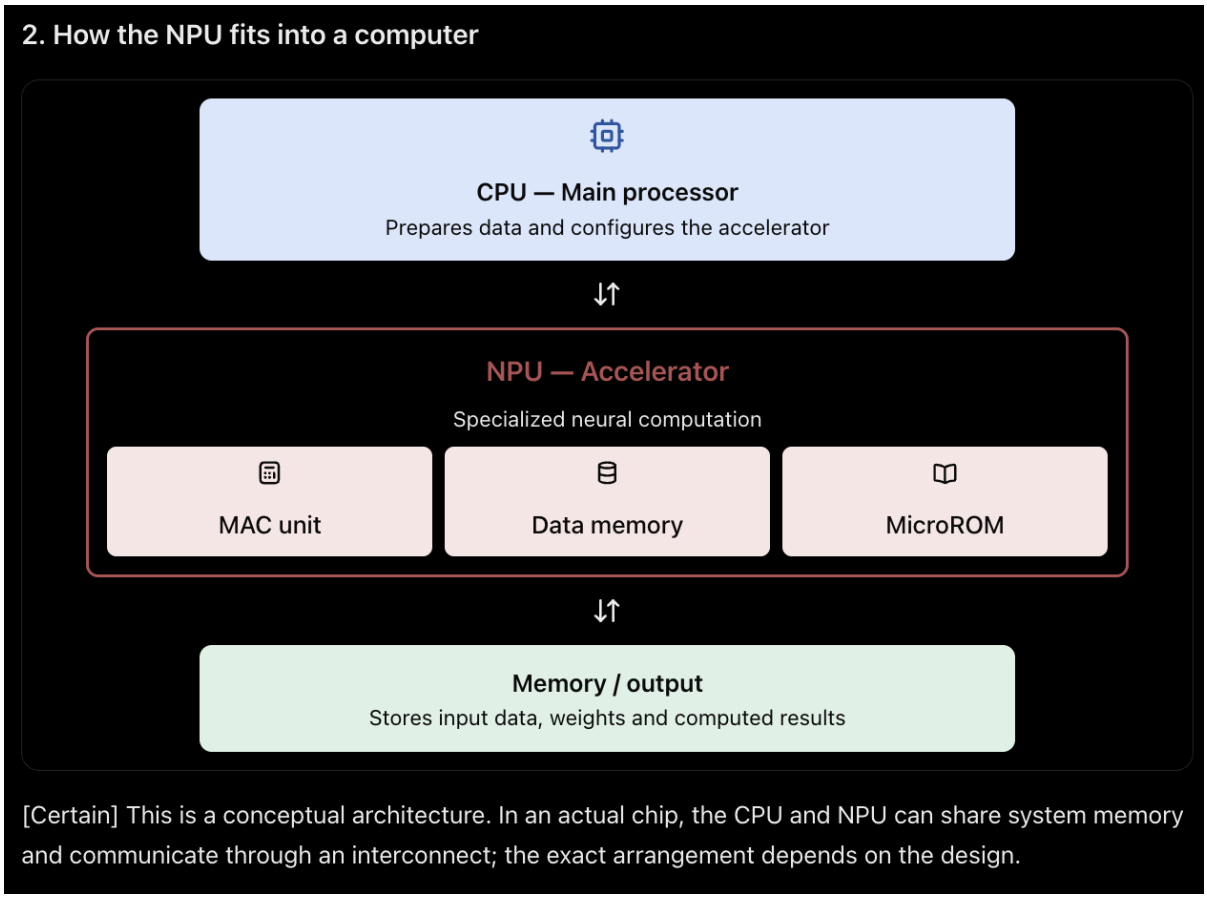


Accelerator

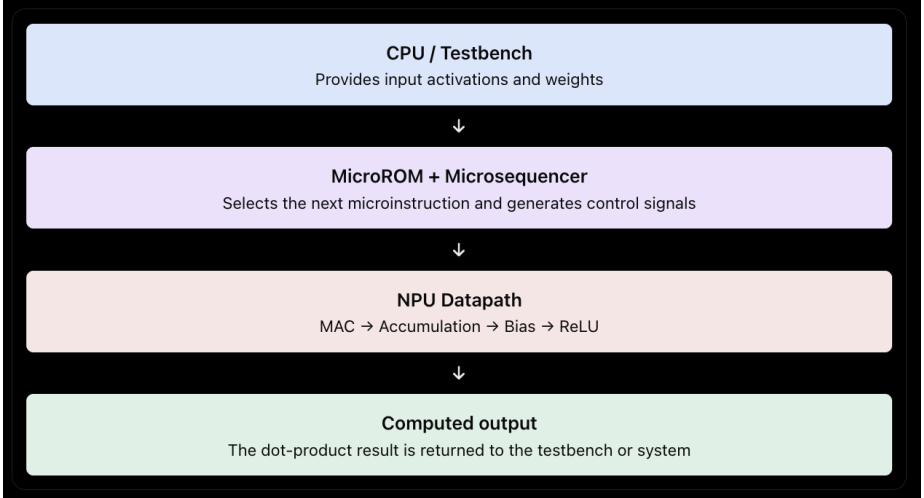


- **CPU: Handles many different tasks — program execution, operating systems, control logic, and general computation.**
- **NPU: Specializes in neural-network operations such as multiply-accumulate (MAC) and matrix multiplication.**

An accelerator does not necessarily replace the CPU. It works alongside it, handling a workload for which it is designed.

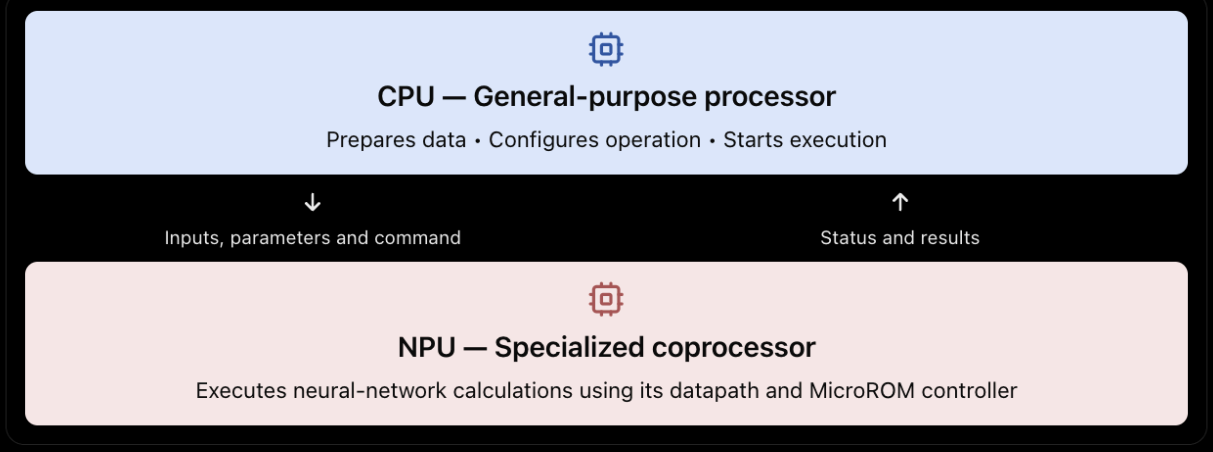


3. How this applies to your MicroROM-based NPU



[Certain] The key architectural distinction is that **the microROM controls the computation; the datapath performs the computation**. Together, they form the core of your microROM-based NPU accelerator.

1. Pictorial explanation

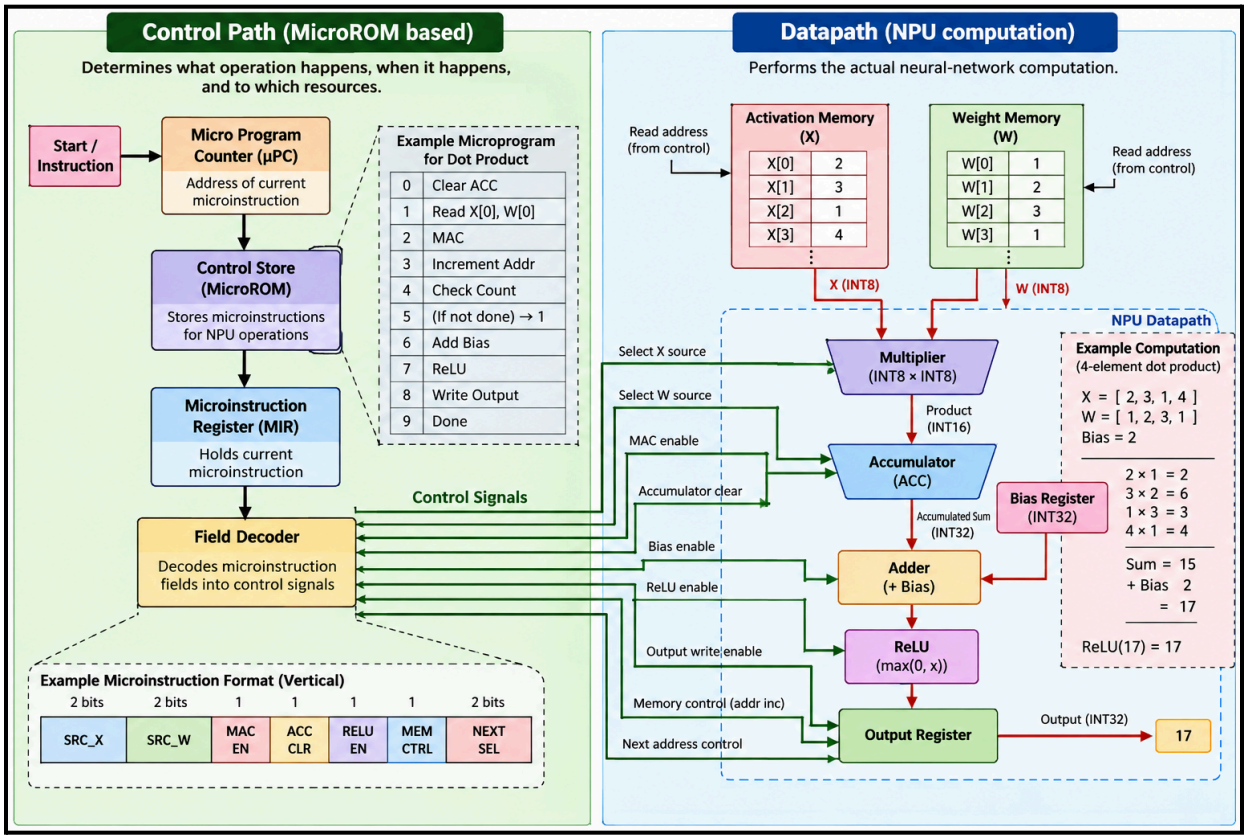


2. What actually happens?

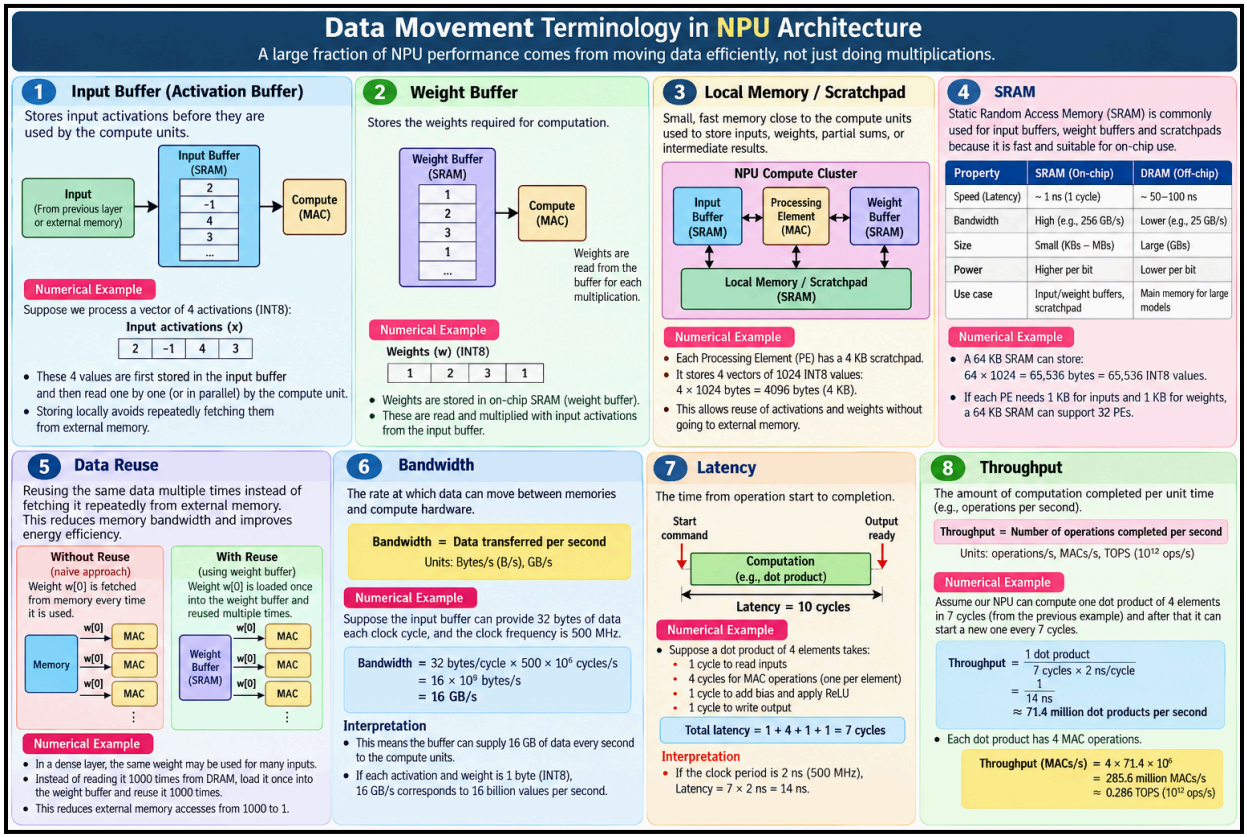
Let's say the CPU wants the NPU to calculate a neural-network dot product.

1. **Prepare inputs:** The CPU or software makes the activation values and weights available to the NPU.
2. **Configure:** The CPU specifies parameters such as vector length, memory addresses and operation mode.
3. **Start:** The CPU issues a command or writes a start register.
4. **Compute:** The NPU executes the operation. Its microsequencer fetches microinstructions from the MicroROM, which controls the datapath.
5. **Complete:** The NPU signals completion, for example through a status register, interrupt or handshake.
6. **Use the result:** The CPU reads the result or uses it in the next stage of the program.

Control Path and Datapath :



Data Movement Terminology in NPU Architecture :



Parallelism in NPU Architectures

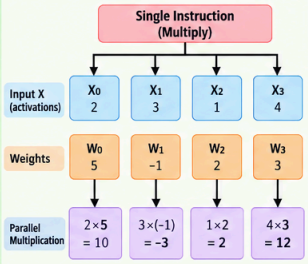
Parallelism in NPU Architectures – Key Concepts with Numerical Examples

1. SIMD (Single Instruction, Multiple Data)

- A single instruction is applied to multiple data elements simultaneously.
- The same operation is performed in parallel on different data.

Concept

Instead of executing one multiply for one pair of elements, execute the same multiply on many pairs at once.



All 4 multiplications are done in parallel with the same instruction.

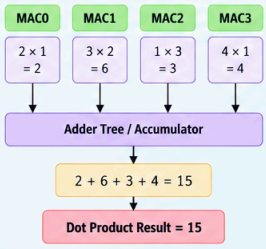
2. Vector MAC (Multiple MACs Operating in Parallel)

- Multiple MAC (Multiply-Accumulate) units work in parallel.
- Each MAC handles one pair (X_i, W_i) and accumulates its result.
- Used for vector dot product, matrix-vector multiplication, etc.

Example: 4-way Vector MAC for Dot Product

Compute: $y = \sum (x_i \times w_i)$

$x = [2, 3, 1, 4]$ $w = [1, 2, 3, 1]$

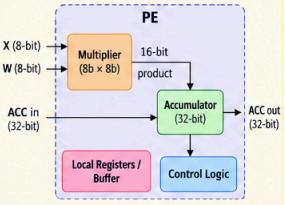


Here 4 MACs operate in parallel, and their results are accumulated to produce the final dot product.

3. Processing Element (PE)

- A Processing Element (PE) is a small compute unit, typically containing:
  - MAC unit
  - Local registers / buffers
  - Control logic (optional)
- Many PEs can work in parallel to increase throughput.

Typical PE Architecture



Numerical Example (single PE operation)

Current ACC = 10
X = 3 W = −4
Product = 3 × (−4) = −12
New ACC = 10 + (−12) = −2

One PE performs one MAC operation per cycle.
An array of PEs can perform many MACs in parallel.

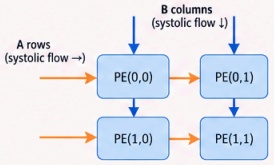
4. Systolic Array

- A structured 2D array of PEs where data flows in a rhythmic (systolic) manner between neighboring PEs.
- Suited for matrix multiplication and large neural network layers.
- Each PE processes data locally and passes partial results to the next PE.

Example: 2 × 2 Systolic Array for Matrix Multiplication

Compute $C = A \times B$

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$ $C = A \times B$



Numerical Resplt

$C = A \times B = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \times \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$

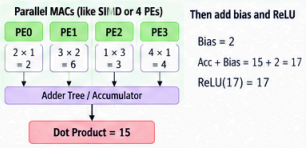
Each PE computes a partial sum and passes it to the next PE. After several cycles, the entire matrix multiplication result is produced.

Comparison of Parallelism Concepts

Concept	What is Parallelized?	Typical Use	Example
SIMD	Same instruction on many data elements	Vector operations, element-wise ops	4 multiplications in parallel
Vector MAC	Multiple MAC units	Dot product, matrix-vector multiply	4 MACs accumulate in parallel
PE	Single PE performs one MAC	Building block for larger arrays	One PE: 3 × (−4) → −12, ACC update
Systolic array	Array of PEs with structured data flow	Matrix multiplication, CNN layers	2 × 2 array computing C = A × B

End-to-End Example: Vector Dot Product Using 4 PEs

Inputs:
$x = [2, 3, 1, 4]$
$w = [1, 2, 3, 1]$



Then add bias and ReLU

Bias = 2
Acc + Bias = 15 + 2 = 17
ReLU(17) = 17

Key Takeaways

1 SIMD	Same operation, many data elements.
2 Vector MAC	Multiple MAC units working in parallel.
3 Processing Element (PE)	A small compute unit with MAC, registers and control.
4 Systolic Array	An array of PEs with structured data movement, ideal for matrix multiplication and neural layers.